

INSTITUTO TECNOLÓGICO DE COSTA RICA

Escuela de Ingeniería en Computación

Curso: Lenguajes de Programación

Sistema de Gestión de la Copa Mundial FIFA 2026

Proyecto 4 — Programación Orientada a Objetos con Ruby on Rails

Profesor: Bryan Tomas Hernández Sibaja

Elaborado por:

Josimar Araya Smith

Justin Lacayo Picado

Federick Fernández Calderón

Fecha: 29 de junio de 2026

Índice

1. Descripción general
2. Enlace de GitHub
3. Pasos de instalación
4. Manual de usuario
5. Arquitectura lógica utilizada
6. Explicación del funcionamiento
7. Diagrama de clases del sistema implementado

1. Descripción general

El proyecto consiste en una aplicación web desarrollada con Ruby on Rails para administrar el formato de la Copa Mundial FIFA 2026. El sistema permite registrar selecciones, administrar grupos, generar los partidos de fase de grupos, registrar resultados, recalcular tablas de posiciones, determinar clasificados, gestionar la fase eliminatoria y mostrar el podio final del torneo.

El torneo está modelado con 48 selecciones distribuidas en 12 grupos de 4 equipos. Cada grupo genera 6 partidos, para un total de 72 partidos en fase de grupos. Clasifican los dos primeros lugares de cada grupo y los ocho mejores terceros, formando un cuadro de 32 equipos.

Fase	Estado implementado	Componentes principales
Fase de grupos	Completa	GroupFixtureGenerator, StandingsCalculator, MatchesController
Clasificación a eliminatoria	Completa	QualificationService, BracketGenerator, AutoBracketService
Fase eliminatoria	Completa	MatchResultProcessor, KnockoutAdvancer, vistas de bracket
Resultados finales	Completa	FinalResultsService, TournamentStatusService, vista champion
Documentación y pruebas	Completa	README, documento técnico, prueba flow_test.rb

2. Enlace de GitHub

Repositorio general del curso: <https://github.com/ItsJustStyles/Lenguajes-de-programacion>

Carpeta específica del Proyecto 4: <https://github.com/ItsJustStyles/Lenguajes-de-programacion/tree/main/Proyecto%204>

La aplicación Rails se ubica en: Proyecto 4/mundial_fifa_2026/

3. Pasos de instalación

3.1 Requisitos previos

Herramienta	Versión / indicación
Ruby	3.2.3, según .ruby-version y Gemfile
Rails	7.1.x
Bundler	Instalar con <code>gem install bundler</code> si no existe
Base de datos	SQLite 3
Sistema operativo recomendado	Linux, macOS o Windows con truquillos (Comandos)

3.2 Instalación (para Ubuntu / WSL)

```
sudo apt update
sudo apt install -y ruby-full ruby-dev build-essential sqlite3 libsqlite3-dev
ruby -v
gem install bundler
```

3.3 Clonar el repositorio y entrar al proyecto

```
git clone https://github.com/ItsJustStyles/Lenguajes-de-programacion.git
cd "Lenguajes-de-programacion/Proyecto 4/mundial_fifa_2026"
```

3.4 Instalar dependencias

```
bundle install
```

3.5 Crear base de datos, migrar y cargar datos iniciales

```
bin/rails db:setup
# Si la base ya existía:
bin/rails db:migrate db:seed
```

3.6 Ejecutar el sistema

```
bin/rails server
```

Luego abrir el navegador en <http://localhost:3000>.

4. Manual de usuario

4.1 Inicio del sistema

Al entrar a la aplicación se muestra el Dashboard principal. Desde ahí se visualiza el estado del torneo, las tablas de posiciones por grupo y los botones de acción disponibles según la fase actual.

Sección / botón	Función
Dashboard	Muestra los 12 grupos, tabla de posiciones, partidos y estado global del torneo.
Grupos	Permite crear, consultar, editar y eliminar grupos.
Selecciones	Permite crear, consultar, editar y eliminar selecciones participantes.
Partidos	Lista partidos por fase y permite registrar resultados.
Cuadro	Muestra la fase eliminatoria desde dieciseisavos hasta la final.
Campeón	Muestra campeón, subcampeón y tercer lugar cuando el torneo termina.

4.2 Generar fase de grupos

Con los grupos y selecciones cargados, el usuario presiona "Generar partidos de grupos (72)". El sistema crea 6 partidos por grupo usando un formato todos contra todos.

4.3 Registrar resultados de grupos

En cada partido se registran los goles del equipo local y visitante. Al guardar, el sistema recalcula automáticamente la tabla del grupo con estos criterios: puntos, diferencia de goles y goles a favor.

Resultado	Puntos asignados
Victoria	3 puntos

Empate	1 punto para cada equipo
Derrota	0 puntos

4.4 Clasificación a eliminatoria

Cuando todos los partidos de grupos quedan finalizados, el sistema calcula automáticamente los 32 clasificados: 24 cupos directos correspondientes a primeros y segundos lugares, más los 8 mejores terceros.

4.5 Fase eliminatoria

La vista “Cuadro” muestra los cruces por ronda. Dieciseisavos, octavos, cuartos, semifinales, partido por tercer lugar y final. Cada partido pendiente con equipos definidos muestra el botón para registrar resultado.

En eliminatoria no puede existir empate final. Si el marcador queda empatado en goles, el formulario habilita los campos de penales. El sistema valida que los penales tengan un ganador antes de guardar el resultado.

Ronda completada	Acción automática del sistema
Dieciseisavos	Genera los 8 partidos de octavos.
Octavos	Genera los 4 partidos de cuartos.
Cuartos	Genera los 2 partidos de semifinales.
Semifinales	Genera la final y el partido por el tercer lugar.
Tercer lugar y final	Permite mostrar el podio final.

4.6 Resultados finales

Cuando la final y el partido por el tercer lugar están completos, la sección “Campeón” presenta el podio con campeón, subcampeón y tercer lugar.

5. Arquitectura lógica utilizada

La aplicación sigue el patrón Modelo-Vista-Controlador propio de Ruby on Rails, complementado con una capa de servicios para encapsular la lógica de negocio. Esta separación evita que los controladores o modelos concentren responsabilidades excesivas.

Capa	Archivos principales	Responsabilidad
Modelos	Group, Team, Match	Representan los datos del dominio y sus relaciones.
Controladores	GroupsController, TeamsController, MatchesController, TournamentController	Reciben acciones del usuario y delegan la lógica a los servicios.
Servicios	StandingsCalculator, QualificationService, BracketGenerator, AutoBracketService, KnockoutAdvancer, FinalResultsService	Implementan reglas de torneo y flujo automático.
Vistas	ERB en app/views	Interfaz gráfica para administrar y consultar el torneo.
Persistencia	SQLite + ActiveRecord	Guarda grupos, equipos, partidos y

		resultados.
--	--	-------------

5.1 Modelos principales

Modelo	Relaciones / datos clave
Group	Tiene muchas selecciones y partidos de grupo. Expone standings para ordenar la tabla.
Team	Pertenece a un grupo. Guarda estadísticas de fase de grupos y banderas de clasificación/eliminación.
Match	Representa partidos de grupos y eliminatoria. Guarda fase, estado, equipos, goles y penales.

5.2 Servicios principales

Servicio	Responsabilidad
GroupFixtureGenerator	Genera los 72 partidos de grupos, 6 por cada grupo.
StandingsCalculator	Recalcula puntos, goles, diferencia, victorias, empates, derrotas y posiciones.
QualificationService	Selecciona primeros, segundos y ocho mejores terceros.
AutoBracketService	Al terminar grupos, genera automáticamente los dieciseisavos.
BracketGenerator	Ordena clasificados por siembra y crea el cuadro inicial.
MatchResultProcessor	Registra resultados, valida penales y dispara efectos secundarios.
KnockoutAdvancer	Avanza ganadores de ronda y crea final/tercer lugar desde semifinales.
FinalResultsService	Consulta campeón, subcampeón, tercer lugar y estado final.
TournamentStatusService	Centraliza el estado global para dashboard y vistas.

5.3 Aplicación de SOLID

Principio	Aplicación en el proyecto
SRP	Cada servicio tiene una responsabilidad concreta: calcular tabla, clasificar, generar bracket, avanzar eliminatoria o consultar podio.
OCP	El flujo puede extenderse agregando servicios o nuevas reglas sin reescribir controladores completos.
DIP	Los controladores dependen de objetos de servicio de alto nivel, no de consultas mezcladas en la interfaz.
DRY	La lógica de clasificación, podio y avance se centraliza y no se duplica en vistas.

6. Explicación del funcionamiento

6.1 Flujo completo del torneo

1. El seed crea los 12 grupos y las 48 selecciones iniciales.
2. El usuario genera los partidos de grupos mediante GroupFixtureGenerator.
3. Cada resultado se procesa con MatchResultProcessor.
4. StandingsCalculator recalcula la tabla del grupo afectado.
5. AutoBracketService detecta si todos los grupos terminaron; si es así, QualificationService marca clasificados y BracketGenerator crea dieciseisavos.
6. Durante eliminatoria, KnockoutAdvancer crea la ronda siguiente al completarse la ronda actual.
7. Después de semifinales, se crea la final y el partido por el tercer lugar.

8. FinalResultsService consulta el podio final para mostrarlo en la vista champion.

6.2 Validación de penales

En fase eliminatoria, si los goles del local y visitante son iguales, el resultado sólo se acepta cuando los penales existen y no están empatados. Esta regla evita que un partido eliminatorio quede sin ganador.

```
if match.knockout? && home_goals == away_goals
  # Deben existir home_penalties y away_penalties.
  # Además, uno debe ser mayor que el otro.
end
```

6.3 Avance automático de rondas

El avance automático depende de que todos los partidos de una ronda estén completados. Los ganadores se toman en orden de cuadro y se emparejan de dos en dos para construir la ronda siguiente.

7. Diagrama de clases del sistema implementado

El siguiente diagrama resume los modelos principales y los servicios que coordinan el flujo del Mundial.

Diagrama lógico de clases y servicios principales

